Implement c program on knapsack problem using greedy method

```
#include <stdio.h>

// A structure to represent an item in the knapsack
struct Item {
    int weight;
    int value;
};

// Function to swap two items
void swap(struct Item *a, struct Item *b) {
    struct Item temp = *a;
    *a = *b;
    *b = temp;
}

// Function to sort items based on value/weight ratio
void sortItemsByValuePerWeight(struct Item items[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            // Calculate value-to-weight ratios
            float r1 = (float)items[j].value / items[j].weight;
            float r2 = (float)items[j + 1].value / items[j + 1].weight;

            // If the next item has a higher value-to-weight ratio, swap
            if (r1 < r2) {
                swap(&items[j], &items[j + 1]);
            }
        }
    }
}

// Function to calculate the maximum profit using the Greedy method (Fractional Knapsack)
float fractionalKnapsack(int capacity, struct Item items[], int n) {
    // Sort items by their value-to-weight ratio
    sortItemsByValuePerWeight(items, n);

    float totalValue = 0.0;  // Total value accumulated in the knapsack
    int currentWeight = 0;   // Current weight of the knapsack

    // Loop through the sorted items
    for (int i = 0; i < n; i++) {
        // If the item can be added entirely without exceeding the capacity
        if (currentWeight + items[i].weight <= capacity) {
            currentWeight += items[i].weight;
            totalValue += items[i].value;
        }
        // If the item cannot be added entirely, add a fraction of it
        else {
            int remainingCapacity = capacity - currentWeight;
            totalValue += (items[i].value * ((float)remainingCapacity / items[i].weight));
            break;  // Knapsack is full, so exit
        }
    }
```

```
    return totalValue;
}

int main() {
    int n, capacity;

    // Input the number of items
    printf("Enter the number of items: ");
    scanf("%d", &n);

    struct Item items[n];

    // Input the weight and value of each item
    for (int i = 0; i < n; i++) {
        printf("Enter weight and value of item %d: ", i + 1);
        scanf("%d %d", &items[i].weight, &items[i].value);
    }

    // Input the knapsack capacity
    printf("Enter the capacity of the knapsack: ");
    scanf("%d", &capacity);

    // Calculate the maximum value we can obtain
    float maxValue = fractionalKnapsack(capacity, items, n);

    // Output the result
    printf("Maximum value we can obtain = %.2f\n", maxValue);

    return 0;
}
```